

Porównanie formatów danych w grafowych sieciach neuronowych (GNNs) oraz ich wydajności obliczeniowej w bibliotekach GNN: PyTorch Geometric (PyG) oraz Deep Graph Library (DGL)

Patryk Graban, Błażej Kardynał, Bartosz Bycul

University of Technology Gdansk - WFTiMS

Abstract

W naszym projekcie zajmiemy się szczegółową analizą i porównaniem formatów danych stosowanych w grafowych sieciach neuronowych (GNNs) oraz ich wpływu na wydajność obliczeniową w dwóch wiodących bibliotekach: PyTorch Geometric (PyG) oraz Deep Graph Library (DGL).

GNN (Graph Neural Network) czyli forma sieci neuronowej reprezentowana w postaci grafu opisana za pomocą węzłów oraz krawędzi. GNN działają poprzez iteracyjne aktualizowanie stanów węzłów na podstawie informacji od sąsiadujących węzłów. Każdy węzeł agreguje informacje od swoich sąsiadów, przetwarzając je za pomocą funkcji propagacji, co pozwala na uchwycenie lokalnych i globalnych struktur grafu.

Projekt obejmuje badanie, jakie formaty danych są najczęściej używane w GNNs, takich jak listy krawędzi, macierze sąsiedztwa oraz reprezentacje rzadkie (np. CSR, COO). Porównamy ich zalety i wady w kontekście reprezentacji grafów, efektywności pamięciowej oraz łatwości implementacji różnych algorytmów.

Przeanalizujemy również funkcjonalności obu bibliotek oraz ich techniki optymalizacji w celu zwiększenia wydajności obliczeniowej. Przeprowadzimy testy wydajnościowe dla różnych typów grafów i modeli GNNs w obu bibliotekach, porównując czas obliczeń, zużycie pamięci oraz skalowalność.

Celem projektu jest opracowanie szczegółowego porównania formatów danych używanych w GNNs oraz wypracowanie rekomendacji dotyczących wyboru odpowiednich formatów danych i bibliotek GNN w zależności od specyficznych wymagań aplikacji.

Keywords: Graph Neural Networks, PyTorch Geometric, Deep Graph Library, Coordinate Matrix, Compressed Sparse Row

1. Wprowadzenie

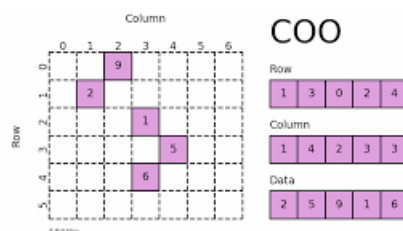
W ostatnich latach grafowe sieci neuronowe (GNNs) zdobyły znaczną popularność dzięki swojej zdolności do modelowania danych strukturalnych, takich jak sieci społeczne, molekularne, czy drogowe. Jednym z kluczowych aspektów efektywności GNNs jest sposób reprezentacji i przetwarzania danych grafowych. W naszym projekcie zajmiemy się porównaniem różnych formatów danych stosowanych w GNNs oraz ich wpływu na wydajność obliczeniową w dwóch popularnych bibliotekach: PyTorch Geometric (PyG) oraz Deep Graph Library (DGL).

2. Formaty danych w GNNs

W naszym projekcie będziemy używali pojęcia macierzy rzadkiej, czyli macierzy której większość elementów stanowią elementy zerowe. Macierz będzie rzadka jeśli jej stosunek NNZ (Number of Non Zero) do rozmiaru macierzy będzie mniejszy niż 0.5. Przechowywanie danych o elementach zerowych jest nieoptymalne, więc będziemy rozważali równe podejścia na przechowywanie tych informacji. W naszym projekcie najbardziej skupimy się na macierzach COO i CSR.

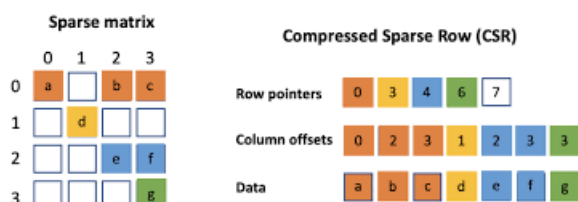
2.1. Coordinate Matrix (COO)

Format COO (Coordinate Matrix) jest prostym i elastycznym sposobem na przechowywanie macierzy rzadkich. Zamiast jednej zwartej struktury, używa trzech tablic: jednej dla indeksów wierszy, jednej dla indeksów kolumn i jednej dla wartości niezerowych. Ten format jest wygodny do budowania macierzy kawałek po kawałku, ponieważ łatwo można dodawać nowe elementy. Jednakże, ponieważ elementy nie są uporządkowane, operacje arytmetyczne oraz mnożenie przez wektory są mniej efektywne. COO jest często używany jako format przejściowy do tworzenia macierzy przed konwersją na bardziej wydajny format, taki jak CSR lub CSC.



2.2. Compressed Sparse Row (CSR)

Format CSR (Compressed Sparse Row) jest idealny do pracy z macierzami rzadkimi, szczególnie kiedy często operujemy na wierszach. W CSR dane są przechowywane w trzech tablicach: jedna przechowuje wszystkie niezerowe wartości, druga zawiera indeksy kolumn dla tych wartości, a trzecia wskazuje początek każdego wiersza w tych tablicach. Dzięki temu można szybko uzyskać dostęp do całych wierszy, co jest bardzo efektywne przy mnożeniu macierzy przez wektory. Format CSR oszczędza pamięć, ale dodawanie nowych elementów może być trudne, ponieważ może wymagać przestawienia całej struktury. Format ten jest szeroko stosowany w obliczeniach naukowych i uczeniu maszynowym.



3. Message-passing i operacje

3.1. Message-passing

Trzyetapowy proces przekazywania informacji między węzłami w sieci neuronowej, zaczynając od wysłania przez każdy węzeł informacji o swoim stanie do każdego ze swoich sąsiednich węzłów. Następuje wtedy proces agregacji informacji poprzez zastosowanie funkcji o niezmiennej permutacji uzyskując ten sam wynik niezależnie od ułożenia się elementów. Proces kończy się uaktualnieniem się cech węzłów poprzez poprzednie i uzyskane w agregacji informacje.

3.2. Operacje SCATTER i GATHER

Stanowią one rdzeń mechanizmu przekazywania wiadomości w GNN. Podczas każdej iteracji lub warstwy GNN, węzły najpierw wysyłają swoje informacje do sąsiadów (SCATTER), a następnie każdy węzeł aktualizuje swój stan na podstawie zebranych informacji od sąsiadów (GATHER). Ten iteracyjny proces pozwala GNN uchwycić zarówno lokalne, jak i globalne struktury grafu, umożliwiając potężne uczenie reprezentacji dla różnych zadań opartych na grafach.

4. Biblioteki

PyTorch Geometric (PyG) i Deep Graph Library (DGL) to dwie wiodące biblioteki do implementacji GNNs, każda z unikalnymi cechami i optymalizacjami.

PyG

Oferuje intuicyjny interfejs do pracy z danymi grafowymi, wspiera różne formaty danych i techniki optymalizacji, takie jak zrównoleglanie obliczeń.

DGL

Oparta na frameworkach takich jak PyTorch i TensorFlow, również oferuje wsparcie dla różnych formatów danych grafowych i optymalizacji, z naciskiem na skalowalność i wydajność.

5. Aktualny stan projektu i jego przyszłe przedsięwzięcia

Uruchomiliśmy skrypty sieci oparte na modelach GCN (Graph Convolutional Network) i GAT (Graph Attention Network), które opierały się na macierzach rzadkich COO badając dokładność tych modeli na zbiorze danych. Model GCN został również przekonwertowany, żeby obierał się na formacie macierzy CSR, dokonując podobnych testów. Następnym naszym celem będzie zaimplementowanie MessagePassing na COO i CSR w pythonie przy użyciu zewnętrznej biblioteki oraz pomiar czasu i peak memory usage dla E2E COO/CSR x CPU/GPU x [GCN/GAT/GraphSage/PNA]

